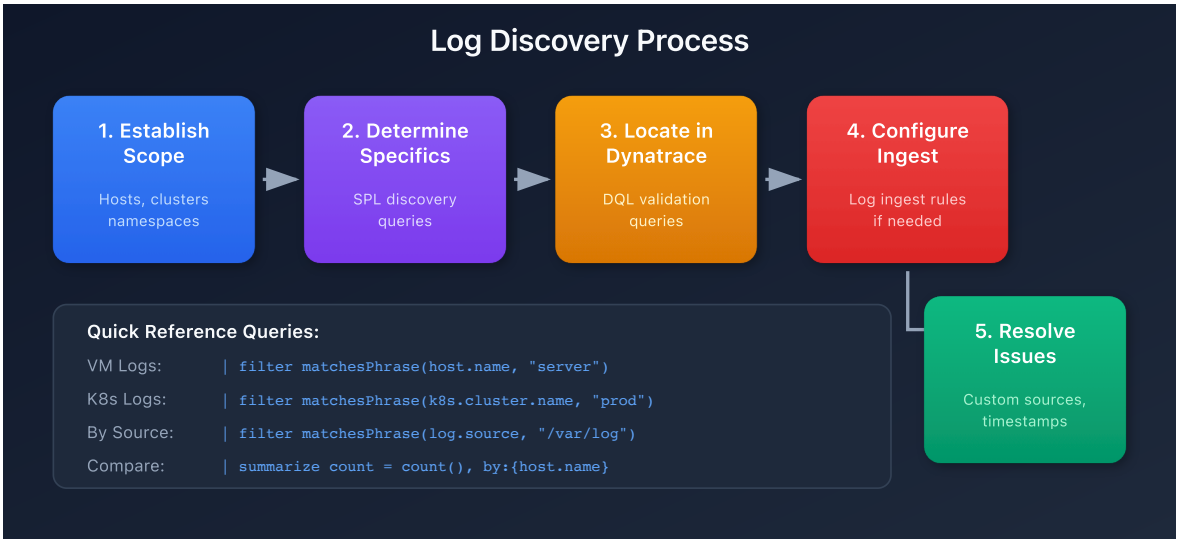
S2D-02: Locating Logs in Dynatrace

> **Series:** S2D | **Notebook:** 2 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Overview

Before migrating Splunk objects such as dashboards, alerts, or reports, it is essential to verify that the log data referenced in these objects is available in Dynatrace.

This notebook describes the process of determining where the required logs can be found, then validating whether they are present in Dynatrace.



Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail
Permissions	`logs.read`, `settings.read`
Splunk Access	Ability to run queries against source indexes
Infrastructure Knowledge	Understanding of application deployment topology

Learning Objectives

By the end of this notebook, you will be able to:

1. Establish the infrastructure scope for log migration
2. Use Splunk queries to identify specific log sources
3. Validate log availability in Dynatrace using DQL
4. Configure log ingest rules when data is missing
5. Troubleshoot common log ingestion issues

Step 1: Establishing Scope

When approaching a new application for migration, the first task is determining where the logs required for monitoring are located. This often coincides with the application's infrastructure, but not always.

Common Log Locations

Environment Type	Log Location Identifier
Virtual Machines	Host or host group
Kubernetes	Cluster, namespace, deployment
Cloud Functions	Cloud provider, function name
Containers	Container name, image

Gathering Scope Information

Information about application scope can come from:

1. **Management Zones** - If configured, these often define application boundaries
2. **Application Teams** - Direct knowledge of deployment topology
3. **CMDB/Inventory** - Infrastructure documentation
4. **Existing Splunk Queries** - Filters used in current dashboards

Step 2: Determining Specifics in Splunk

Not all logs within an application's footprint are required for every dashboard and alert. Conversely, dashboards and alerts may reference logs outside the expected footprint.

Splunk Discovery Queries

Run these queries in Splunk to understand the specific log sources being used:

Record Count by Host

```
index=[your_index] | stats count by host
```

Record Count by Source

```
index=[your_index] | stats count by source, sourcetype
```

Record Count by Kubernetes Attributes

```
index=[your_index] | stats count by kubernetes_cluster, kubernetes_namespace,
kubernetes_deployment_name
```

> **Tip:** If your Splunk environment doesn't use index-based organization, substitute the filters from your actual queries before the `stats` command.

Step 3: Locating Logs in Dynatrace

To determine whether required logs are present in Dynatrace, use DQL queries with the relevant filters from your Splunk discovery.

Search by Host Name

```
// Search logs by host name
// Replace 'your-host-name' with the actual host from Splunk
fetch logs
| filter matchesPhrase(host.name, "your-host-name")
| summarize count = count(), by:{log.source}
| sort count desc
```

Search by Log Source Path

```
// Search by specific log file path
// Replace path with your application's log location
fetch logs
| filter matchesPhrase(log.source, "/var/log/application")
| summarize count = count(), by:{host.name}
| sort count desc
```

Search by Kubernetes Attributes

```
// Search by Kubernetes cluster and namespace
fetch logs
| filter matchesPhrase(k8s.cluster.name, "production-cluster")
| filter matchesPhrase(k8s.namespace.name, "ecommerce")
| summarize count = count(), by:{k8s.deployment.name, k8s.pod.name}
| sort count desc
```

Compare Counts with Splunk

Run equivalent time-bounded queries in both platforms to compare log volumes:

```
// Total log count for the last hour (compare with Splunk)
fetch logs, from:now()-1h
| filter matchesPhrase(host.name, "your-host-name")
| summarize total_count = count()
```

Step 4: Configuring Log Ingest

If the required logs are not available in Dynatrace, you need to configure log ingest rules.

Ingest Rule Hierarchy

Level	Scope	Best For
Environment	All hosts	Organization-wide policies
Host Group	Specific host groups	Application-specific rules
Host	Single host	Special cases

> **Recommendation:** Use host group-level rules for application granularity without per-host overhead.

Log Ingest Rule Properties

Rules can filter logs based on:
- Host name patterns
- Log file name or path
- Kubernetes attributes
- Log level
- Process name
- Content patterns

Documentation Reference

See Log Ingest Rules for complete configuration options.

Step 5: Resolving Discrepancies

If logs are still not appearing after configuring ingest rules, investigate these common issues:

Custom Log Sources

If your application writes to non-standard log locations, you may need to configure Custom Log Sources.

OneAgent Log Monitoring

Verify that log content access is enabled on the OneAgent. The `oneagentctl` command can check this:

```
oneagentctl --get-log-content-access
```

> **Note:** Host-level settings supersede environment configuration. If log monitoring is disabled on the OneAgent, environment-level rules will not apply.

Timestamp and Splitting Issues

If logs appear but counts differ or content seems incorrect, you may need Timestamp and Splitting Rules to properly parse multi-line logs or custom timestamp formats.

Troubleshooting Checklist

Symptom	Possible Cause	Solution
No logs from host	Ingest rule missing	Add ingest rule for host/host group
No logs from file	Custom log source needed	Configure custom log source
Some logs missing	OneAgent disabled	Enable log content access
Wrong log count	Timestamp parsing	Configure timestamp rules
Multi-line broken	Splitting rules	Configure splitting rules

Validation Query Template

Use this template to create a comprehensive validation query for your application:

```
// Comprehensive log validation template
// Modify filters to match your application
fetch logs, from:now()-24h
| filter matchesPhrase(host.name, "app-server") OR
matchesPhrase(k8s.namespace.name, "app-namespace")
| summarize
    total_count = count(),
    error_count = countIf(loglevel == "ERROR"),
    warn_count = countIf(loglevel == "WARN"),
    by:{log.source}
| sort total_count desc
```

Next Steps

Once you've validated that required logs are available in Dynatrace, proceed to **S2D-03: SPL to DQL Translation** to begin converting your Splunk queries.

References

- Log Ingest Rules
- Custom Log Sources
- Log Content Access
- Timestamp Configuration
